

Programmation réseau



Avant propos

- Un **objet** est une **instance d'une classe**
 - Tout en Python est un objet : nombres, chaînes, listes, fonctions, classes elles-mêmes
 - Un objet combine **des attributs** et **des méthodes**
 - Attributs : Variables associées à l'objet (ex : x.valeur)
 - Méthodes : Fonctions associées à l'objet (ex : x.afficher())
 - Objet = un struct qui héberge des pointeurs sur fonction !
 - La classe = le typedef du struct
 - Une instance de classe : une variable (objet) de cette classe (de ce type d'objet)

Avant propos

```
1  ▼ class Personne:
2  ▼     def __init__(self, nom, age):
3      self.nom = nom          # attribut
4  └     self.age = age        # attribut
5
6  ▼     def sePresenter(self):  # méthode
7  └     print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")
8
9  # Création d'un objet
10 p = Personne("Alice", 30)
11 p.nom = "toto" # accès à l'attribut nom
12 p.sePresenter() # Appelle la méthode
13
```

Socket

- On été introduit dans l'UNIX de Berkeley
- Un modèle permettant la communication inter processus
 - Au sein d'une machine
 - Entre 2 machines
- **Socket** est un point de communication pour établir le dialogue entre deux processus
- Dans le cadre du réseau, il faut :
 - une **adresse IP** (identifie la machine)
 - un **port** (identifie le service sur cette machine)
 - un **protocole** (TCP ou UDP)

Protocole : couche transport

■ TCP (Transmission Control Protocol)

- Orienté connexion (comme un appel téléphonique)
- Fiable : garantie que tous les paquets arrivent, dans le bon ordre
- Plus lent mais robuste
- Exemples : HTTP, FTP, messagerie instantanée

■ UDP (User Datagram Protocol)

- Sans connexion (comme envoyer une carte postale)
- Pas de garantie de livraison ni d'ordre
- Très rapide, faible surcharge réseau
- Exemples : jeux en ligne, visioconférence, streaming audio/vidéo

Cycle de vie d'une communication

TCP

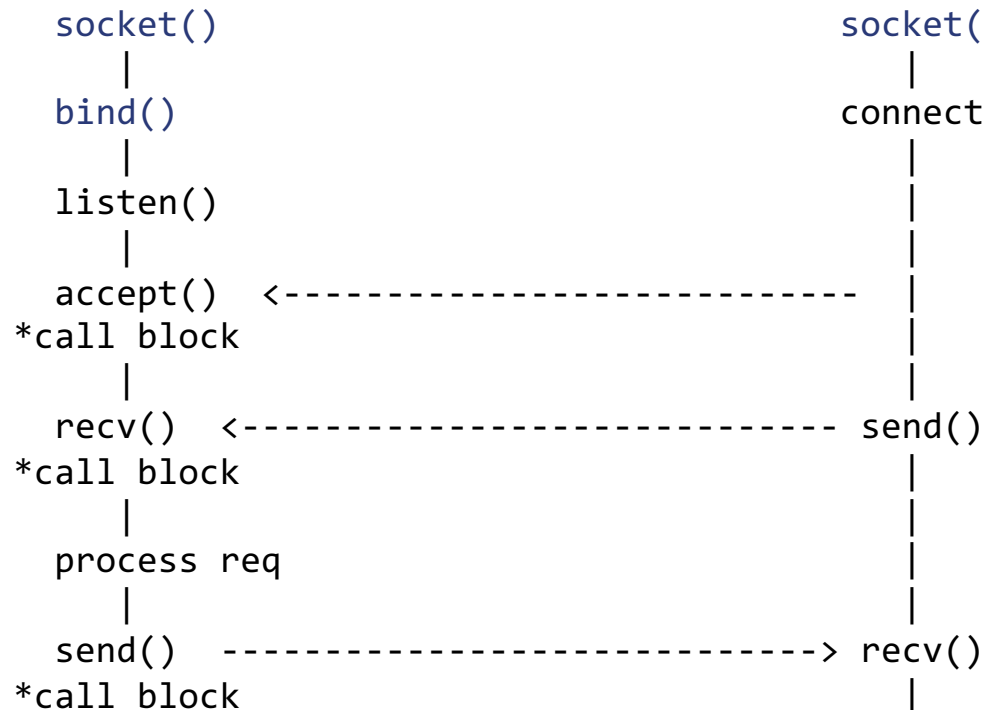
- Le serveur écoute
 - *bind + listen*
- Le client se connecte
 - *connect*
- Les deux échangent des messages
 - *send, recv*
 - Répéter n fois
- Ils ferment la connexion
 - *close*

UDP (pas de connexion)

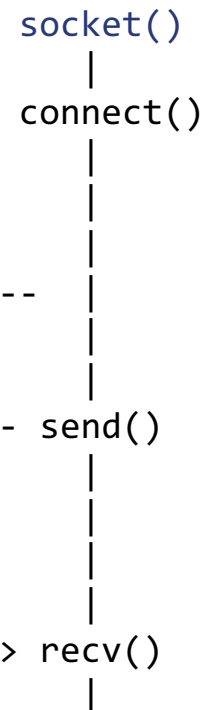
- Le serveur écoute
 - *bind*
- Le client envoie directement un datagramme
 - *sendto*
- Le serveur peut répondre
 - *sendto*

Cycle de vie d'une communication

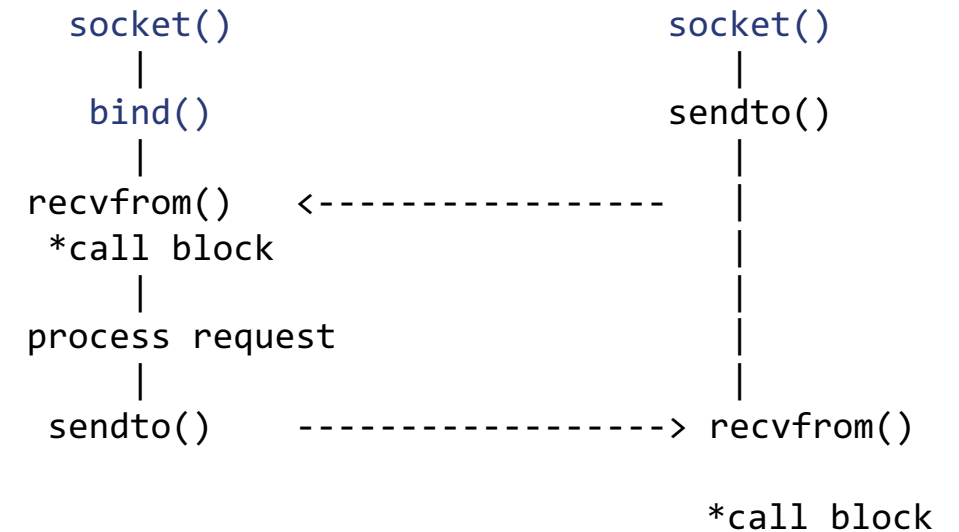
SERVER (TCP)



CLIENT (TCP)



SERVER (UDP)



CLIENT (UDP)

*call block : *l'appel de la fonction va bloquer l'exécution jusqu'à ce que l'événement attendu se produise*

Serveur : attente de connexion

- Ecouter le socket : `listen`
 - Paramètre : Nombre maximum de connexions en attente avant que le système refuse les nouvelles

```
s.listen(5)
```

 - Obligatoire pour du TCP
 - Transforme le socket en socket "écoutant" (listening socket)
- Accepter une connexion
 - Retourne :
 - le socket connecté créer, il permet de dialoguer avec le client
 - un tuple avec l'adresse IP et le port de connexion du client

```
client, client_infos = srv.accept()
```

Dans la pratique

- En Python : module socket
- L'API est très proche entre TCP et UDP
- La façon de se connecter/envoi diffère
- Dans la suite du cours
 - coder un serveur *qui accepte un seul client*,
 - puis on le transforme en multi-clients

C / Python / Java

■ Même concepts de base

- socket → bind → listen → accept → send/recv → close

- C (bas niveau)

- API POSIX
- Manipulation de struct sockaddr_in et buffers mémoire
- Très verbeux, mais contrôle total
- Proche du matériel (système d'exploitation)

- Java (orienté objet)

- Classes Socket et ServerSocket
- Utilise InputStream / OutputStream au lieu de buffers
- Plus haut niveau, intégré dans l'écosystème objet

- Python (niveau intermédiaire)

- API presque identique à C
- Classe socket
- Code concis et lisible

Client + serveur : établir une connexion

- Le client se connecte au serveur
- Il nous faut connaître :
 - Le **nom d'hôte** qui identifie une machine
 - Le resolver se chargera de convertir le nom de l'hôte en IP
 - Un **numéro de port** compris entre 0 et 65535
 - les numéros entre 0 et 1023 sont réservés pour des services connus
- Exemple : ssh transit.univ-lemans.fr
 - Hôte : transit.univ-lemans.fr
 - Port : 23 (ssh)

```
srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```



Client et serveur

- `Socket(family, type, proto=0)`
 - créer d'une instance socket = un point de communication réseau
 - point de départ de toute communication réseau : TCP, UDP, IPv4, IPv6, etc.

Family

`socket.AF_INET` : IPV4

✓ : Linux, MacOS, Windows,

`socket.AF_INET6` : IPV6

✓ : Linux, MacOS, Windows,

`socket.AF_UNIX` : communication locale par fichier

✓ : Linux, MacOS, ⚠ : Windows (partiel)

...

`socket.AF_BLUETOOTH`

✓ : Linux, MacOS, ⚠ : Windows (partiel)

`socket.AF_CAN` : bus CAN dans l'automobile

✓ : Linux, ✗ : MacOS, Windows

Type

`socket.SOCK_STREAM` : TCP

✓ : Linux, MacOS, Windows,

`socket.SOCK_DGRAM` : UDP

✓ : Linux, MacOS, Windows,

`socket.SOCK_RAW` : paquet IP (IP personnalisé, ICMP)
privilège administrateur

✓ : Linux, ⚠ : MacOS (partiel), Windows (partiel)

...

Proto, souvent laissé à 0 car déduit de family et type

`AF_INET + SOCK_STREAM = socket.IPPROTO_TCP`

`AF_INET + SOCK_DGRAM = socket.IPPROTO_UDP`

`AF_INET + SOCK_RAW = socket.IPPROTO_ICMP`



Serveur : attente de connexion

- Methode **bind()** : lier un socket à une adresse locale
 - Paramètre : tuple (*nom_hote*, *port*)
 - Nom d'hôte : doit appartenir à la machine
 - Une adresse IP d'une interface de la machine : '192.168.0.10', '127.0.0.1'
 - Toutes les adresses IP de la machine : '0.0.0.0' ou ''
 - Un nom = 'localhost', socket.gethostname()
 - Fait appel au resolver
 - Numéro de port (entre 0 et 65535)
 - 0 : le système choisit un numéro de libre
 - < 1024 : besoin du privilège root

Serveur : attente de connexion

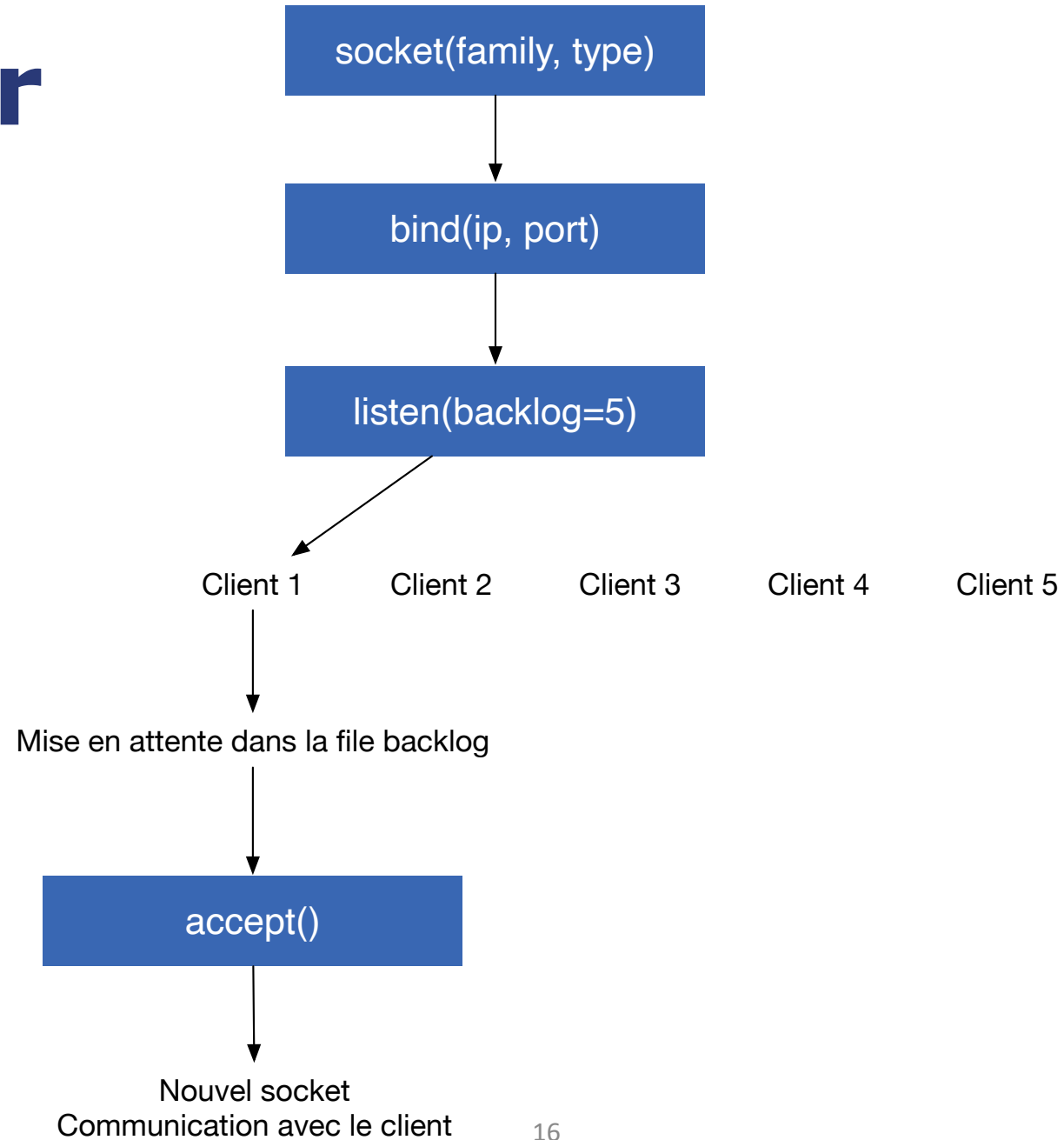
- Methode `bind()` : lier un socket à une adresse locale
 - Comment savoir si un port est libre ?
 - → essayer de l'ouvrir !

```
1  import socket
2  from typing import Tuple, Any
3
4  def open_port(port: int, host: str = "0.0.0.0") -> Tuple[bool, Any]:
5      s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6      try:
7          s.bind((host, port))
8          return True, s # la socket est ouverte et peut être utilisée
9      except OSError:
10         s.close()
11         return False, None
```

Serveur : attente de connexion

- Ecouter le socket : methode `listen(backlog: int)`
 - `backlog` : Nombre maximum de connexions en attente avant que le système refuse les nouvelles
 - Obligatoire pour du TCP, n'existe pas en UDP
 - Transforme la socket en un socket "écoutant" (listening socket)
- Accepter une connexion : methode `comm, addr = accept()`
 - bloque jusqu'à ce qu'un client se connecte
 - Fonctionne si le socket est en mode écoute
 - Retourne :
 - `comm` : nouvel socket connecté (il permet de dialoguer avec le client)
 - `addr` : tuple avec l'adresse IP et le port de connexion du client
 - **le socket original reste en écoute pour d'autres connexions**

Serveur



Client : connexion

- Création du client

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

- Connexion au serveur

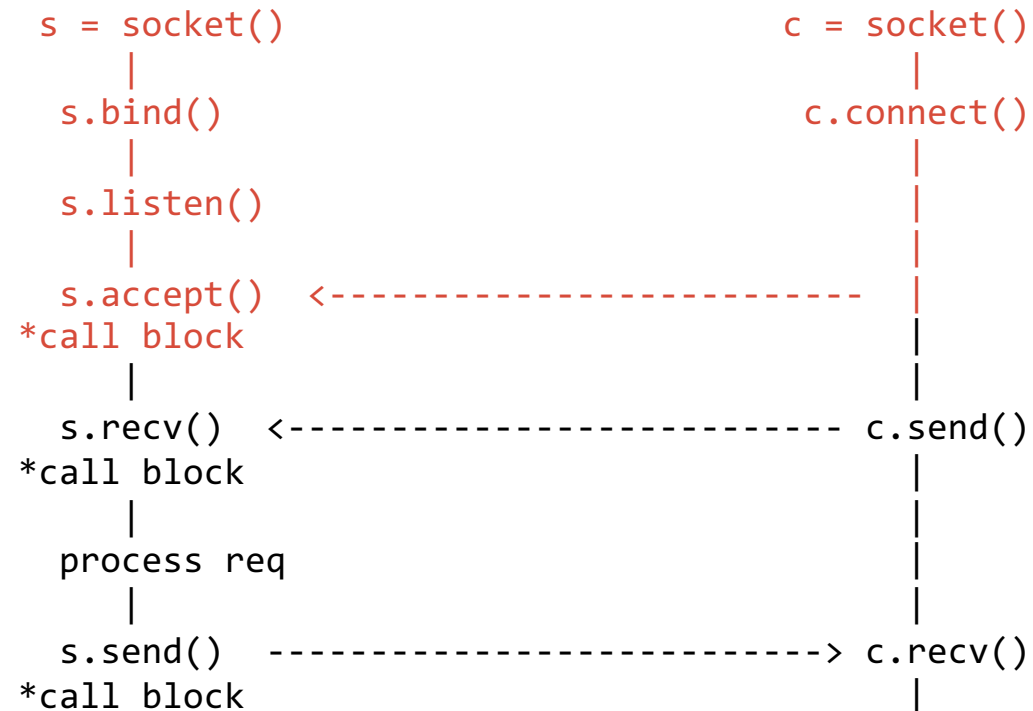
- Tente d'établir une connexion TCP avec le serveur
- Bloque jusqu'à ce que la connexion soit acceptée par le serveur ou qu'une erreur survienne
 - le serveur doit être en écoute (`listen()`) pour accepter la connexion
- Paramètre : nom du serveur, numéro de port du serveur

```
client.connect(('localhost', 12800))
```

Client / Serveur

SERVER (TCP)

CLIENT (TCP)



Réseau : envoyer

- Par défaut les sockets sont un flux de bytes
- TCP
 - Methode `send(buffer)`
 - Tente d'envoyer les données sur la socket TCP
 - Retourne le nombre d'octets effectivement envoyés
 - Peut envoyer moins que la totalité si la socket est saturée
 - Methode `sendall(buffer)`
 - Tente d'envoyer tous les octets
 - Bloque jusqu'à ce que tout soit envoyé ou qu'une erreur se produise
- UDP
 - Methode `sendto(buffer, adress)`
 - la socket n'a pas besoin d'être connectée
 - Chaque message est indépendant : pas de flux continu comme TCP
 - retourne le nombre d'octets effectivement envoyés

Réseau : recevoir

- Par défaut les sockets sont un flux de bytes
- TCP
 - Methode `data = recv(bufsize)`
 - La socket doit être connectée (client) ou issue d'un `accept()` (serveur)
 - Bloque jusqu'à ce que des données arrivent
 - Retourne un objet bytes
 - Peut retourner moins d'octets que demandés
- UDP
 - Methode `data, addr = recvfrom(bufsize)`
 - bloque jusqu'à ce qu'un datagramme arrive
 - Data reçu, adresse du client (IP, port) qui a envoyé le datagramme
 - Chaque datagramme est indépendant, et aucune connexion n'est nécessaire

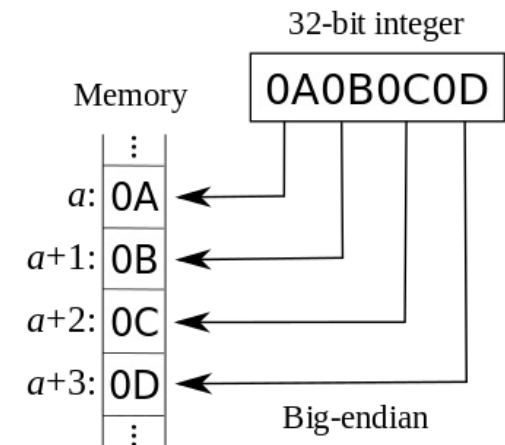
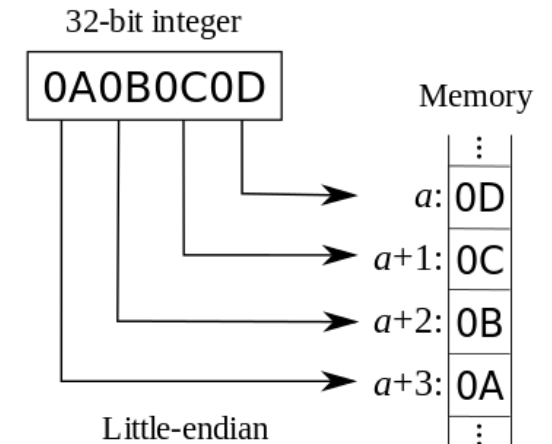
Codage de l'information

- Chaîne de caractères
 - Attention
 - au codage
 - aux fins de ligne → CR (\r) et/ou LF (\n)
 - Gestion de la taille de la chaîne
 - Utilise encode() pour passer d'une chaîne Unicode à des octets
 - Le décodage inverse se fait avec decode()

```
1 text = "Hello"
2 b = text.encode("utf-8") # b'Hello'
3 s = b.decode("utf-8") # "Hello"
```

Codage de l'information

- Entier : attention au endianness et conversion en bytes
 - Pour les entiers 32 bits (cas python)
 - `struct.pack('!l',x)` et `struct.unpack('!l',x)`
 - ! → format réseau
 - L → int sur 4 octets
- Float : conversion en bytes
 - `struct.pack('!d',x)` et `struct.unpack('!d',x)`



Lecture de n bytes

```
def recvall(sock, count):  
    buf = b''  
    while count:  
        newbuf = sock.recv(count)  
        if not newbuf: return None  
        buf += newbuf  
        count -= len(newbuf)  
    return buf
```



Lecture de float, int et str

```
def recvint(sock):  
    data = recvall(sock, 4)  
    if not data:  
        return None  
    v, = struct.unpack('!l', data)  
    return v  
  
def recvstr(sock):  
    n = recvint(sock)  
    if not n:  
        return None  
    return recvall(sock, n).decode('utf8')
```

Ecriture de float, int et str

```
def sendint(sock, x):  
    sock.sendall(struct.pack('!l',x))
```

```
def sendstr(sock, x):  
    sendint(sock, len(x))  
    sock.sendall(bytes(x, 'utf8'))
```



Réseau : client

```
import socket
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
s.connect((TCP_IP, TCP_PORT))
```

```
msg=input("msg: ")  
while msg != 'stop':  
    sendstr(s, msg)  
    print('send: ', msg)  
    data = recvstr(s)  
    print('recv: ', data)  
    msg=input("msg: ")  
s.close()
```



Réseau : 1 serveur 1 client

```
import socket

srv= socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.bind(('', 8888)) # '': toutes les interfaces disponibles
srv.listen(5)

try:
    while True:
        cli, info = srv.accept()
        print("Connected from", cli_info)
        while True:
            data= cli.recv(1024)
            if not data:
                break
            cli.sendall(data)
        cli.close()
        print("Disconnected from", cli_info)
finally:
    srv.close()
```

Réseau : 1 serveur n clients

- En réseau, un serveur peut devoir gérer plusieurs connexions simultanées
 - sockets bloquantes → un client bloque l'application
- Méthodes classiques :
 - Threads / Process → lourd et pas encore étudié
- Solution :
 - programmation non bloquante
 - multiplexage avec selectors

Réseau : 1 serveur n clients

- Multiplexage d'entrées/sorties
 - Surveiller plusieurs sockets à la fois sans bloquer
 - Savoir quelles sockets sont prêtes à lire, écrire, ou présentent une erreur
- Principes de selectors
 - Fournit une abstraction portable :
 - Windows → `select()`,
 - Linux → `epoll()`
 - macOS → `kqueue()`
 - Les sockets sont enregistrées pour des événements spécifiques.
 - On récupère les sockets prêtes à agir via `select()` sur le **Selector**

■ Utilisation

- Créer un Selector
- Enregistrer une socket

```
1 import selectors
2 sel = selectors.DefaultSelector()
```

```
4 sel.register(socket_obj, selectors.EVENT_READ | selectors.EVENT_WRITE, data=my_data)
```

- Socket_obj : un socket ou stdin
- EVENT_READ → lecture possible
- EVENT_WRITE → écriture possible
- data → objet associé, une fonction (callback) pour suivre l'état ou identifier la socket
- Boucle d'événements
 - select() retourne les sockets prêtes.
 - On peut gérer plusieurs clients dans une seule boucle sans threads

Réseau : 1 serveur n clients

```
1  import socket
2  import selectors
3
4  sel = selectors.DefaultSelector() # Création du sélecteur
5
6  # --- Socket serveur ---
7  server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Création d'une socket TCP
8  server.bind(('0.0.0.0', 9000)) # Bind sur toutes les interfaces et port 9000
9  server.listen() # Mettre en écoute
10 server.setblocking(False) # Non bloquant
11 sel.register(server, selectors.EVENT_READ, data=None) # On surveille la lecture (nouvelle connexion)
```

Réseau : 1 serveur n clients

```
11 sel.register(server, selectors.EVENT_READ, data=None) # On surveille la lecture (nouvelle connexion)
12 # --- Boucle événementielle ---
13 while True:
14     events = sel.select(timeout=None) # Attend qu'au moins une socket soit prête
15     for key, mask in events:
16         if key.data is None: # Nouvelle connexion sur la socket serveur
17             conn, addr = server.accept() # Acceptation de la connexion
18             conn.setblocking(False) # Non bloquant pour le client
19             sel.register(conn, selectors.EVENT_READ | selectors.EVENT_WRITE, data={"addr": addr, "out_buffer": b""})
20             # Enregistrement pour lecture et écriture, avec buffer de sortie
21         else:
22             conn = key.fileobj # Socket client
23             data = key.data # Données associées à la socket
24
25             # --- Lecture possible ---
26             if mask & selectors.EVENT_READ:
27                 try:
28                     recv_data = conn.recv(1024) # Lecture des données
29                     if recv_data:
30                         data["out_buffer"] += recv_data # Stockage dans le buffer pour renvoyer (echo)
31                     else:
32                         sel.unregister(conn) # Retirer du sélecteur
33                         conn.close() # Fermer la socket
34                 except ConnectionResetError: # Client fermé brutalement
35                     sel.unregister(conn)
36                     conn.close()
37
38             # --- Écriture possible ---
39             if mask & selectors.EVENT_WRITE:
40                 if data["out_buffer"]: # Si le buffer n'est pas vide
41                     sent = conn.send(data["out_buffer"]) # Envoi des données
42                     data["out_buffer"] = data["out_buffer"][sent:] # Suppression des octets envoyés
```

Réseau : 1 serveur n clients

```
1 import socket
2 import selectors
3
4 sel = selectors.DefaultSelector() # Création du sélecteur
5
6 # --- Socket serveur ---
7 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Création d'une socket TCP
8 server.bind(('0.0.0.0', 9000)) # Bind sur toutes les interfaces et port 9000
9 server.listen() # Mettre en écoute
10 server.setblocking(False) # Non bloquant
11 sel.register(server, selectors.EVENT_READ, data=None) # On surveille la lecture (nouvelle connexion)
12 # --- Boucle événementielle ---
13 while True:
14     events = sel.select(timeout=None) # Attend qu'au moins une socket soit prête
15     for key, mask in events:
16         if key.data is None: # Nouvelle connexion sur la socket serveur
17             conn, addr = server.accept() # Acceptation de la connexion
18             conn.setblocking(False) # Non bloquant pour le client
19             sel.register(conn, selectors.EVENT_READ | selectors.EVENT_WRITE, data={"addr": addr, "out_buffer": b""})
20             # Enregistrement pour lecture et écriture, avec buffer de sortie
21         else:
22             conn = key.fileobj # Socket client
23             data = key.data # Données associées à la socket
24
25             # --- Lecture possible ---
26             if mask & selectors.EVENT_READ:
27                 try:
28                     recv_data = conn.recv(1024) # Lecture des données
29                     if recv_data:
30                         data["out_buffer"] += recv_data # Stockage dans le buffer pour renvoyer (echo)
31                 except:
32                     sel.unregister(conn) # Retirer du sélecteur
33                     conn.close() # Fermer la socket
34             except ConnectionResetError: # Client fermé brutalement
35                 sel.unregister(conn)
36                 conn.close()
37
38             # --- Écriture possible ---
39             if mask & selectors.EVENT_WRITE:
40                 if data["out_buffer"]: # Si le buffer n'est pas vide
41                     sent = conn.send(data["out_buffer"]) # Envoi des données
42                     data["out_buffer"] = data["out_buffer"][sent:] # Suppression des octets envoyés
```



Coté client

- Le client simple peut se passer de selectors
 - envoie un message puis attend une réponse
- utiliser `recv()` bloquant après le `send()`

```
1  import socket
2
3  # --- Création et connexion du client ---
4  client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Création d'une socket TCP
5  client.connect(('127.0.0.1', 9000)) # Connexion au serveur pédagogique
6
7  # --- Envoi d'un message ---
8  message = "Bonjour serveur !"
9  client.sendall(message.encode()) # Envoi du message encodé en bytes
10
11 # --- Réception de la réponse ---
12 data = client.recv(1024) # Lecture bloquante, attend le message du serveur
13
14 # --- Fermeture de la connexion ---
15 client.close()
16
```

Coté client

- Le client ne peut être bloqué
 - Interface graphique
 - Client multi-serveurs ou multi-sockets
 - Communication asynchrone continue
 - recevoir et envoyer des messages en même temps

```
1 import socket
2 import selectors
3
4 sel = selectors.DefaultSelector() # Création du sélecteur
5
6 # --- Création de la socket client ---
7 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Socket TCP
8 client.setblocking(False) # Non bloquant pour la connexion
9
10 try:
11     client.connect(('127.0.0.1', 9000)) # Connexion au serveur
12 except BlockingIOError:
13     pass # Connexion non bloquante, exception normale temporaire
14
15 # Buffer de sortie (message à envoyer)
16 out_buffer = b"Bonjour serveur !"
17
18 # Enregistrement de la socket pour lecture et écriture
19 sel.register(client, selectors.EVENT_READ | selectors.EVENT_WRITE, data={"out_buffer": out_buffer})
20
21 # --- Boucle événementielle ---
22 while True:
23     events = sel.select(timeout=1) # Attend qu'une socket soit prête
24     for key, mask in events:
25         sock = key.fileobj
26         data = key.data
27
28         # --- Écriture possible ---
29         if mask & selectors.EVENT_WRITE:
30             if data["out_buffer"]: # Si on a des données à envoyer
31                 sent = sock.send(data["out_buffer"]) # Envoi non bloquant
32                 data["out_buffer"] = data["out_buffer"][sent:] # Supprime les octets envoyés
33
34         # --- Lecture possible ---
35         if mask & selectors.EVENT_READ:
36             try:
37                 recv_data = sock.recv(1024) # Lecture non bloquante
38                 if recv_data:
39                     sel.unregister(sock) # On a reçu la réponse, plus besoin de surveiller
40                     sock.close() # Fermeture propre
41                     exit(0) # Fin du client pédagogique
42             except:
43                 else:
44                     print("Serveur fermé")
45                     sel.unregister(sock)
46                     sock.close()
47                     exit(0)
48     except BlockingIOError:
49         pass # Pas de données disponibles
```

Résumé

■ Concepts clés

- Sockets : interface pour communication TCP/UDP entre machines
 - Family : AF_INET → IPv4
 - Types de sockets :
 - SOCK_STREAM → TCP (fiable, orienté connexion)
 - SOCK_DGRAM → UDP (non fiable, sans connexion)
- Bind / Listen / Accept → côté serveur pour gérer les connexions entrantes
- Connect / Send / Recv → côté client pour communiquer avec le serveur

Résumé

- **Problème** : blocage avec `recv()/send()` sur plusieurs clients
- **Solutions** :
 - Threads / Process → simple mais moins scalable
 - **Selectors** → gestion simultanée de plusieurs sockets dans une seule boucle